Cloud Company

CAHIER D'ANALYSE

Orchestration de ressources multi-VM et multi-réseaux avec Heat et Auto-scaling

Rédiger par :

Ngouo Franck Leonel

MOUDIO ABEGA Laurent Stéphane

Supervisé par : M NGUIMBUS Emmanuel

PLAN DE PRÉSENTATION

1. Introduction

2. Contexte et Objectifs

3. Périmètre Fonctionnel et Exigences

4. Architecture des Templates Avancés et du Scaling

5. Modélisation UML (Conception de la Solution)

6. Conclusion

1. Introduction

Ce présent cahier d'analyse a pour objectif de détailler les besoins et les fonctionnalités du projet visant à améliorer l'orchestration des ressources cloud en utilisant le service Heat d'OpenStack. L'automatisation du déploiement et de la gestion des infrastructures est essentielle dans un contexte informatique dynamique. Ce projet vise à surmonter les limites de la version actuelle de Heat en développant des modèles (templates HOT) avancés et des modules de scaling automatique personnalisés.

2. Contexte et Objectifs

2.1. Contexte et Problématique

OpenStack fournit un ensemble de services pour gérer les ressources de calcul, de stockage et de réseau. Le service Heat permet d’orchestrer ces déploiements via des modèles déclaratifs.

La problématique observée est que l'outil présente des limites qui freinent son adoption dans des scénarios complexes :

- Complexité de modélisation des architectures multi-VM et multi-réseaux. Les templates deviennent longs et difficiles à maintenir.
- Limitation dans la gestion automatique de la scalabilité (montée/descente en charge).

2. Contexte et Objectifs

2.2. Objectifs du Projet

Les objectifs du projet sont de :

- Concevoir des templates Heat complexes capables de décrire et déployer des topologies **multi-VM et multi-réseaux**.
- Développer des **modules Heat personnalisés** pour gérer le **scaling automatique** basé sur la charge (CPU, mémoire, trafic).
- Mettre en place un environnement de test basé sur **DevStack** pour la validation des déploiements.
- Mesurer les **performances** et l'**efficacité** de la solution proposée.

3. Périmètre Fonctionnel et Exigences

3.1. Besoins Fonctionnels (BF)

- BF1 - Déploiement Complet : Le système doit permettre la création automatisée d’architectures complètes via un seul template.
- BF2 - Scaling Intelligent : La solution doit exécuter le scaling automatique des instances en fonction de la charge mesurée (ex: CPU, mémoire).
- BF3 - Modularité : La conception des templates doit permettre la réutilisation des sous-templates pour améliorer la modularité du code.
- BF4 - Monitoring : La solution doit s'intégrer avec un outil de monitoring externe pour récupérer les métriques de charge.

3. Périmètre Fonctionnel et Exigences

3.2. Besoins Non Fonctionnels (BF)

- **BNF1 - Performance :** Le déploiement d’une infrastructure complexe doit être réalisé **plus rapidement** que les méthodes manuelles.
- **BNF2 - Sécurité :** L’infrastructure déployée doit assurer l'**isolation réseau** et une gestion rigoureuse de l'**authentification**.
- **BNF3 - Maintenabilité :** Le code des templates et des modules personnalisés doit être **modulaire et bien documenté**.

4. Architecture des Templates Avancés et du Scaling

4.1. Structure des Templates HOT Avancés

L'architecture repose sur des Templates Imbriqués. Contrairement à l'approche Monolithique initiale Nous avons ici:

. **Un Template Racine:** Orchestre les dépendances.

. **Des Sous-Templates:** Définissent les ressources atomiques.

4. Architecture des Templates Avancés et du Scaling

4.2. Implémentation du Scaling Dynamique (Architecture détaillé)

l’architecture est définis comme suite:

1. Source : Les VMs exposent un endpoint /metrics.

2. Collecteur (système externe): Récupère les métriques et évalue des règles d'alerte

3. Transmission: En cas d'alerte, le collecteur l’envoie vers l'Adaptateur.

4. l’Adaptateur de Métriques: Ce composant est un middleware développé spécifiquement. Il reçoit l’alerte Valide l'authenticité de la source (Sécurité).Transforme l'alerte en signal Heat

5. Action (Heat Engine) : Reçoit le signal et exécute l'action de scaling définie (Scale Up/Down).

5. Modélisation UML

Diagramme de Cas d'Utilisation :

Acteurs :

1. Administrateur Cloud OpenStack

- **Type :** Humain
- **Description :** Responsable du déploiement, de la supervision et de la maintenance des infrastructures cloud
- **Rôle :**
  - Initie les déploiements d'architectures complexes
  - Surveille les performances et métriques
  - Consulte les logs pour le diagnostic et l'audit
  - Configure les politiques de scaling

5. Modélisation UML

Diagramme de Cas d'Utilisation :

Acteurs :

2. Système de Monitoring Externe

- **Type :** Système autonome
- **Description :** Collecte et fourniture de données de charge en temps réel.
- **Rôle :**
  - Collecte et analyse les métriques de performance en temps réel
  - Déclenche les actions de scaling horizontal (scale-in/scale-out)
  - Applique les politiques de scaling définies
  - Interagit avec l'API Heat pour modifier les infrastructures

5. Modélisation UML

Diagramme de Cas d'Utilisation :

Acteurs :

2. Développeur Cloud / Architecte

- **Type :** Humain
- **Description :** Conception et implémentation des modèles d'infrastructure
- **Rôle :**
  - Création et maintenance des Templates HOT Modulaires
  - définition des politiques de Scaling (Custom Resources

5. Modélisation UML

Diagramme de Cas d'Utilisation :

Cas d'Utilisation :

CU1 : Déployer et Mettre à Jour une Architecture Modulaire.

CU2 : Configurer une Politique de Scaling

CU3 : Intégrer un de Métriques Externe.

CU4 : Exécution du Scaling Automatique

5. Modélisation UML

Description textuelle des cas d’utilisation

Cas d'utilisation 1 : Déployer une architecture complète

Titre :	Déployer et Mettre à Jour une Architecture Modulaire
Objectif :	Permettre à l'administrateur de déployer où mettre à jour une infrastructure multi-niveaux complète en une seule commande
Résumé :	L'administrateur utilise le template Heat principal pour déployer l'ensemble de l'architecture (load balancer, niveaux web, application, base de données..)
Acteur :	Administrateur Cloud OpenStack

Présupposé	Précondition(s)	Post condition(s)
• L'environnement OpenStack est opérationnel • Les images système sont disponibles • Les quotas de ressources sont suffisants	• L'administrateur est authentifié avec des droits suffisants • Les templates Heat sont accessibles et valides	• La pile Heat est créée avec statut CREATE_COMPLETE • Toutes les ressources (VM, réseaux, volumes) sont déployées • L'application est accessible

Scénario nominal

1. L'administrateur se connecte à la CLI OpenStack ou au dashboard Horizon
2. Il télécharge les templates Heat depuis le dépôt Git
3. Il adapte le fichier de paramètres params.env selon ses besoins
4. Il exécute la commande : openstack stack create -t master_stack.yaml -e params.env mon-app-prod
5. Heat charge le template et orchestre le déploiement des stacks imbriqués
6. L'administrateur surveille la progression via openstack stack list
7. Une fois complété, il récupère l'IP en sortie.
8. En cas de mise à jour **(Update)** : Heat compare l'état actuel et la nouvelle définition, et applique les changements (ex: ajout d'un sous-réseau) sans détruire l'existant non impacté.

Scénario alternatif

A1. Si les paramètres sont invalides, Heat rejette la création avec un message d'erreur

. L'administrateur corrige les paramètres et relance la commande

A2. Si le déploiement échoue sur une ressource

. Heat rollback partiellement la stack

. L'administrateur consulte les logs détaillés pour identifier l'erreur

. Il corrige la cause (quota, image manquante, etc.) et relance

Cas d'utilisation 2 : Configurer une Politique de Scaling Dynamique Personnalisée

Titre :	*Configurer une Politique de Scaling (KPIs Métier)*
Objectif :	*Permettre au Développeur Cloud de définir des règles de mise à l'échelle basées sur des métriques métiers ou de charge non natives.*
Résumé :	*Le Développeur crée une Custom Resource qui lie l'AutoScalingGroup aux seuils de métriques personnalisées définis pour une application spécifique*
Acteur :	*Développeur Cloud*

Présupposé	***Précondition(s)***	***Post condition(s)***
• *Le template Heat existe et est fonctionnel* • *L'adaptateur de métrique est foctionnel*	• *Le Custom Resource Type pour le scaling est enregistré et fonctionnel dans Heat.* • *L'AutoScalingGroup cible est défini.*	• *La Custom Resource est créée.* • *Le Heat Engine surveille les seuils de la métrique personnalisée pour l'AutoScalingGroup ciblé, en attente de notification.*

Scénario nominal

1. *Le Développeur modifie le template pour inclure la ressource* ***Custom::Heat::ScalingPolicy.***
2. *Le template spécifie la métrique cible , le seuil (ex: >500), la période d'observation et l'action* ***(Scale-Up ou Scale-Down).***
3. *Le Développeur déploie la Stack mise à jour.*
4. *Le* ***Heat Engine*** *valide et enregistre la* ***Custom Resource,*** *établissant le lien de déclenchement avec l'adaptateur de métriques externe.*
5. *Le système de surveillance est armé pour notifier Heat lors du franchissement du seuil.*

Scénario alternatif

A1. Métrique Inconnue: *Le nom de la métrique spécifiée dans la Custom Resource n'est pas reconnu par l'adaptateur de métriques externe. Heat Engine retourne un message d'erreur*

A2. Permissions Insuffisantes: *le Développeur n'a pas les permissions nécessaire pour développer ou créer ou modifier une ressource de type* ***Custom::Heat::ScalingPolicy.*** *Heat rejette la création pour non respect de l'authentification*

Cas d'utilisation 3 : Intégrer un Système de Métriques Externe

Titre :	*Surveiller les métriques de performance*
Objectif :	*Établir le pont de communication sécurisé entre l'outil de monitoring et le Heat Engine pour permettre le scaling basé sur la charge réelle*
Résumé :	*L'Administrateur configure les endpoints et l'authentification pour que le système de monitoring puisse communiquer les alertes de Custom Metrics au Heat Engine.*
Acteur :	*Administrateur Cloud, Système de Monitoring Externe*

Présupposé	***Précondition(s)***	***Post condition(s)***
• *La stack du module de scaling est déployé* • *Les agents exporteurs de métriques sont installés sur les instances*	• *L'Adaptateur de Métriques Personnalisées est déployé et opérationnel.* • *Les métriques d'application sont collectées (via l'outil de monitoring).*	• *Le Heat Engine reçoit et traite les notifications de l'outil externe.* • *Le scaling dynamique (CU4) est fonctionnellement prêt à être déclenché.*

Scénario nominal

1. *L'Administrateur déploie l'Adaptateur de Métriques Personnalisées dans le cluster OpenStack.*
2. *L'Administrateur configure les règles d'exportation de métriques dans l'outil de monitoring pour exposer les données critiques vers l'Adaptateur.*
3. *L'Administrateur configure les mécanismes de sécurité (jetons API, groupes de sécurité Neutron) pour garantir l'intégrité et l'isolation réseau des communications*
4. *Le Heat Engine confirme la réception d'un événement de test ou d'une métrique via l'Adaptateur.*

Scénario alternatif

A1. Problème de Sécurité Réseau : *L'isolation réseau bloque la communication entre l'Adaptateur et l'API Heat. L'Administrateur doit ajuster les règles Neutron pour rétablir la connectivité tout en maintenant l'isolation .*

A2. Incompatibilité de Format : *L'outil de monitoring envoie des données de métriques dans un format que l'Adaptateur ne peut pas interpréter. L'Administrateur doit reconfigurer l'outil pour se conformer au schéma attendu par l'Adaptateur.*

Cas d'utilisation 4 : Mettre à l'Échelle Automatiquement

Titre :	*Mettre à l'Échelle Automatiquement une Application*
Objectif :	*Le système doit réagir automatiquement à un changement de charge pour ajuster les ressources de l'AutoScalingGroup, améliorant la performance .*
Résumé :	*Déclenchement automatique d'une action de scaling (Scale-Up ou Scale-Down) suite à la détection d'un dépassement de seuil d'une métrique personnalisée .*
Acteur :	*Système (Heat Engine, Custom Metrics Adapter, Nova, Neutron)*

Présupposé	***Précondition(s)***	***Post condition(s)***
• *L'application et les Ressources sont disponibles* • *Le systèmes de collecte de métriques est opérationnels ainsi que les politiques de scaling*	• *Une Politique de Scaling Dynamique est configurée (CU2).* • *L'application atteint un seuil de charge critique.*	• *Le nombre d'instances VM dans l'AutoScalingGroup est ajusté.* • *Le système revient à un état de charge normale.*

Scénario nominal

1. *La charge sur l'application dépasse le seuil configuré (ex: plus de 3 requêtes actives par pod en moyenne)*
2. *Le Système de Monitoring Externe détecte le dépassement et envoie une alerte normalisée via l'Adaptateur de Métriques.*
3. *Le Heat Engine reçoit l'événement, l'authentifie, et identifie la Custom Resource de scaling associée.*
4. *Le Heat Engine exécute l'action de Scale-Up, demandant à Nova de créer une nouvelle VM et à Neutron d'attacher un port réseau à cette VM.*
5. *Une fois la VM active, elle est intégrée à l'AutoScalingGroup, réduisant la charge moyenne.*

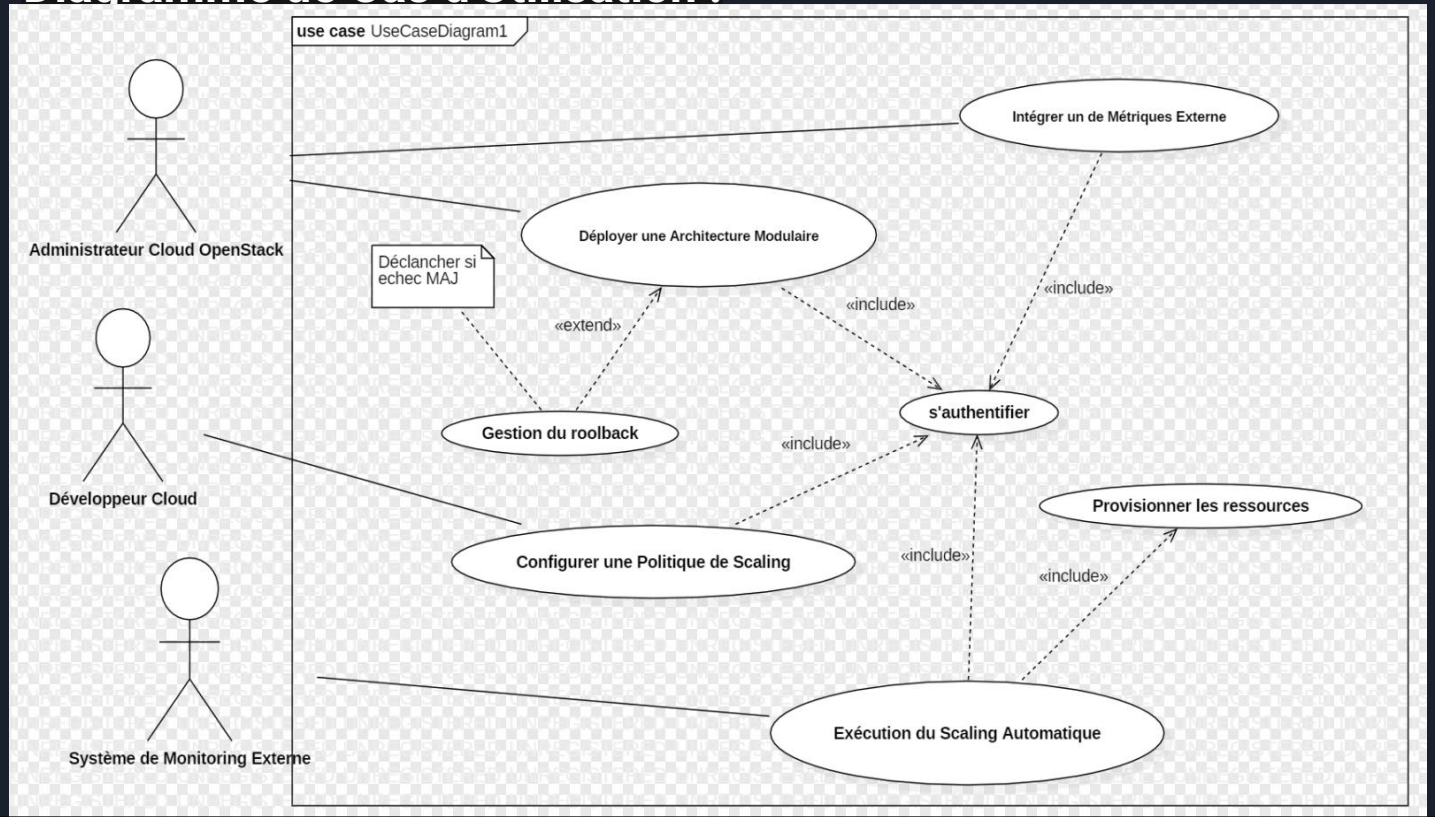
Scénario alternatif

A1. Quota Atteint : *Nova ne peut pas provisionner une nouvelle VM car le quota de l'utilisateur est atteint. Le Heat Engine enregistre un événement d'échec et notifie l'Administrateur, indiquant que l'action de Scale-Up a échoué.*

A2. Échec de Désactivation : *Lors d'un Scale-Down, la VM cible est incapable d'être mise hors service ou son retrait du Load Balancer échoue. Le Heat Engine applique une politique de temporisation et retente l'opération ou marque la ressource pour une intervention manuelle.*

5. Modélisation UML

Diagramme de Cas d'Utilisation :



5. Conclusion

Ce Cahier d'Analyse révisé propose une approche structurée et techniquement robuste pour la conception de la solution d'orchestration avancée au sein d'OpenStack Heat. L'analyse a confirmé deux limitations fondamentales de l'existant : **la complexité et la faible maintenabilité des templates monolithiques (L1) et la rigidité du mécanisme de scaling basé sur des seuils d'infrastructure statiques et non sécurisés (L2).**

La solution technique proposée résout ces défis en imposant deux stratégies d'architecture modernes:

1. **Templates Modulaires et Gestion du Cycle de Vie :** L'adoption de template imbriqué pour le déploiement (CU1) permet de satisfaire les besoins de modularité (BF3) tout en garantissant la maintenabilité future via des stratégies de mises à jour progressives .

1. **Scaling Dynamique Sécurisé par Custom Resources :** L'introduction de Custom Resources (CU2) couplées à un Adaptateur de Métriques Externe (CU3) permet à l'orchestrateur de réagir à la charge réelle et aux KPIs applicatifs (CU4).